FLUTTER SQLITE APPLICATIONS

Lab Sheet – 8 | Complete Question-wise Solutions

This solution set follows the 17 programs listed in the uploaded lab sheet. The examples use Flutter with sqflite and path. Each question includes the core implementation pattern; related questions reuse the same database setup and table structure.

1. Project setup

```
flutter pub add sqflite path
```

Add these imports in the Dart file:

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
```

2. Common SQLite helper

Use this helper as the database foundation. Create the required table for each question in onCreate (the table definitions are shown below).

```
class DBHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final dbPath = await getDatabasesPath();
    _db = await openDatabase(
      join(dbPath, 'lab8_database.db'),
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''CREATE TABLE notes(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          title TEXT, content TEXT)''');
        await db.execute('''CREATE TABLE contacts(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT, phone TEXT)''');
        await db.execute('''CREATE TABLE expenses(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          title TEXT, amount REAL)''');
        await db.execute('''CREATE TABLE tasks(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          task TEXT, completed INTEGER DEFAULT 0)''');
        await db.execute('''CREATE TABLE users(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT, email TEXT UNIQUE, password TEXT)''');
        await db.execute('''CREATE TABLE students(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT, email TEXT, course TEXT)''');
        await db.execute('''CREATE TABLE products(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT, quantity INTEGER, price REAL)''');
        await db.execute('''CREATE TABLE cart(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT, quantity INTEGER, price REAL)''');
      },
    );
    return _db!;
  }

  static Future<int> insert(String table, Map<String, Object?> values) async =>
      (await database).insert(table, values);

  static Future<int> update(String table, Map<String, Object?> values,
      int id) async =>
      (await database).update(table, values,
          where: 'id = ?', whereArgs: [id]);

  static Future<int> delete(String table, int id) async =>
      (await database).delete(table, where: 'id = ?', whereArgs: [id]);

  static Future<List<Map<String, Object?>>> getAll(String table) async =>
      (await database).query(table, orderBy: 'id DESC');
}
```

Q1. Notes application – insert and display notes

```
// Insert a note
await DBHelper.insert('notes', {
  'title': titleController.text.trim(),
  'content': contentController.text.trim(),
```

```
});

// Display notes in a ListView
final notes = await DBHelper.getAll('notes');
ListView.builder(
  itemCount: notes.length,
  itemBuilder: (context, index) {
    final note = notes[index];
    return ListTile(
      title: Text(note['title'].toString()),
      subtitle: Text(note['content'].toString()),
    );
  },
)
```

Q2. Edit an existing note

```
// Call this when a note is tapped. First put its values in the controllers.
titleController.text = note['title'].toString();
contentController.text = note['content'].toString();

// Save edited values
await DBHelper.update('notes', {
  'title': titleController.text.trim(),
  'content': contentController.text.trim(),
}, note['id'] as int);

// Refresh the list after update.
setState(() {});
```

Q3. Delete note by swipe and show SnackBar

```
Dismissible(
  key: ValueKey(note['id']),
  direction: DismissDirection.endToStart,
  background: Container(
    color: Colors.red,
    alignment: Alignment.centerRight,
    padding: const EdgeInsets.all(16),
    child: const Icon(Icons.delete, color: Colors.white),
  ),
  onDismissed: (_) async {
    await DBHelper.delete('notes', note['id'] as int);
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Note deleted successfully')),
    );
    setState(() {});
  },
  child: ListTile(title: Text(note['title'].toString())),
)
```

Q4. Search notes by title

```
Future<List<Map<String, Object?>>> searchNotes(String keyword) async {
  final db = await DBHelper.database;
  return db.query(
    'notes',
    where: 'title LIKE ?',
    whereArgs: ['%$keyword%'],
    orderBy: 'title ASC',
  );
}

// TextField example
TextField(
  decoration: const InputDecoration(
    labelText: 'Search notes by title',
    prefixIcon: Icon(Icons.search),
  ),
  onChanged: (value) async {
    final result = await searchNotes(value);
    setState(() => filteredNotes = result);
  },
)
```

Q5. Contact Book – add and display contacts

```
// Add contact
await DBHelper.insert('contacts', {
  'name': nameController.text.trim(),
  'phone': phoneController.text.trim(),
});

// Load and show contacts
final contacts = await DBHelper.getAll('contacts');
```

```
ListView.builder(
  itemCount: contacts.length,
  itemBuilder: (context, index) => ListTile(
    leading: const Icon(Icons.person),
    title: Text(contacts[index]['name'].toString()),
    subtitle: Text(contacts[index]['phone'].toString()),
  ),
)
```

Q6. Delete contact after AlertDialog confirmation

```
Future<void> confirmDelete(Map<String, Object?> contact) async {
  final yes = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Delete contact?'),
      content: Text('Delete ${contact['name']}?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, false),
          child: const Text('Cancel'),
        ),
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, true),
          child: const Text('Delete'),
        ),
      ],
    ),
  );

  if (yes == true) {
    await DBHelper.delete('contacts', contact['id'] as int);
    setState(() {});
  }
}
```

Q7. Search contacts by name using LIKE

```
Future<List<Map<String, Object?>>> searchContacts(String name) async {
  final db = await DBHelper.database;
  return db.query(
    'contacts',
    where: 'name LIKE ?',
    whereArgs: ['%$name%'],
  );
}

// Call searchContacts(value) from a search TextField's onChanged
// and assign the returned list to the ListView data source.
```

Q8. Expense Tracker – add and display expenses

```
await DBHelper.insert('expenses', {
  'title': titleController.text.trim(),
  'amount': double.tryParse(amountController.text) ?? 0.0,
});

final expenses = await DBHelper.getAll('expenses');
ListView.builder(
  itemCount: expenses.length,
  itemBuilder: (context, index) => ListTile(
    title: Text(expenses[index]['title'].toString()),
    trailing: Text('■${expenses[index]['amount']}'),
  ),
)
```

Q9. Display total expense using SUM

```
Future<double> getTotalExpense() async {
  final db = await DBHelper.database;
  final result = await db.rawQuery(
    'SELECT SUM(amount) AS total FROM expenses',
  );
  return (result.first['total'] as num?)?.toDouble() ?? 0.0;
}

// Display the value
FutureBuilder<double>(
  future: getTotalExpense(),
  builder: (context, snapshot) {
    final total = snapshot.data ?? 0.0;
    return Text('Total Expense: ■${total.toStringAsFixed(2)}');
  },
)
```

Q10. To-Do app – update completed status with Checkbox

```
// Add a task
await DBHelper.insert('tasks', {
  'task': taskController.text.trim(),
  'completed': 0,
});

// Checkbox in each task row
CheckboxListTile(
  title: Text(task['task'].toString()),
  value: (task['completed'] as int) == 1,
  onChanged: (checked) async {
    await DBHelper.update('tasks', {
      'completed': checked == true ? 1 : 0,
    }, task['id'] as int);
    setState(() {});
  },
)
```

Q11. Registration – store name, email and password

```
Future<void> registerUser() async {
  final name = nameController.text.trim();
  final email = emailController.text.trim();
  final password = passwordController.text;

  if (name.isEmpty || email.isEmpty || password.isEmpty) return;

  try {
    await DBHelper.insert('users', {
      'name': name,
      'email': email,
      'password': password,
    });
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Registration successful')),
    );
  } on DatabaseException {
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Email already registered')),
    );
  }
}

// Learning/demo note: real applications must never store plain-text passwords.
// Use a secure authentication service or properly salted password hashing.
```

Q12. Login – validate username and password, then open Home

```
Future<void> login() async {
  final db = await DBHelper.database;
  final result = await db.query(
    'users',
    where: 'email = ? AND password = ?',
    whereArgs: [
      emailController.text.trim(),
      passwordController.text,
    ],
    limit: 1,
  );

  if (!context.mounted) return;
  if (result.isNotEmpty) {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => const HomeScreen()),
    );
  } else {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Invalid email or password')),
    );
  }
}

// HomeScreen is a widget you create for the application's home page.
// For production, use secure authentication; do not compare plain-text passwords.
```

Q13. Validate form before inserting a record

```
final formKey = GlobalKey<FormState>();

Form(
  key: formKey,
  child: Column(
```

```
    children: [
      TextFormField(
        controller: nameController,
        decoration: const InputDecoration(labelText: 'Name'),
        validator: (v) =>
            v == null || v.trim().isEmpty ? 'Enter name' : null,
      ),
      TextFormField(
        controller: emailController,
        decoration: const InputDecoration(labelText: 'Email'),
        validator: (v) {
          if (v == null || !RegExp(r'^[^@]+@[^@]+\.[^@]+$').hasMatch(v)) {
            return 'Enter a valid email';
          }
          return null;
        },
      ),
      TextFormField(
        controller: passwordController,
        obscureText: true,
        decoration: const InputDecoration(labelText: 'Password'),
        validator: (v) => v == null || v.length < 6
            ? 'Password must be at least 6 characters' : null,
      ),
      ElevatedButton(
        onPressed: () async {
          if (formKey.currentState!.validate()) {
            await DBHelper.insert('users', {
              'name': nameController.text.trim(),
              'email': emailController.text.trim(),
              'password': passwordController.text,
            });
          }
        },
        child: const Text('Register'),
      ),
    ],
  ),
)
```

Q14. Student List and Add/Edit Student screens with Navigator

```
// Open Add/Edit screen
final changed = await Navigator.push<bool>(
  context,
  MaterialPageRoute(
    builder: (_) => StudentFormScreen(student: selectedStudent),
  ),
);
if (changed == true) setState(() {});

// Save inside StudentFormScreen.
// If student is null, insert; otherwise update the selected row.
Future<void> saveStudent(Map<String, Object?>? student) async {
  final values = {
    'name': nameController.text.trim(),
    'email': emailController.text.trim(),
    'course': courseController.text.trim(),
  };

  if (student == null) {
    await DBHelper.insert('students', values);
  } else {
    await DBHelper.update('students', values, student['id'] as int);
  }
  if (!context.mounted) return;
  Navigator.pop(context, true);
}

// Student list data comes from DBHelper.getAll('students').
```

Q15. Product Inventory – add, update and delete products

```
// Add
await DBHelper.insert('products', {
  'name': nameController.text.trim(),
  'quantity': int.tryParse(quantityController.text) ?? 0,
  'price': double.tryParse(priceController.text) ?? 0.0,
});

// Update selected product
await DBHelper.update('products', {
  'name': nameController.text.trim(),
  'quantity': int.tryParse(quantityController.text) ?? 0,
  'price': double.tryParse(priceController.text) ?? 0.0,
}, product['id'] as int);

// Delete selected product
```

```
await DBHelper.delete('products', product['id'] as int);

// Display products using DBHelper.getAll('products') and ListView.builder.
```

Q16. Shopping Cart – store items and display total price

```
// Add an item to cart
await DBHelper.insert('cart', {
  'name': productName,
  'quantity': quantity,
  'price': price,
});

// Calculate total = sum of quantity multiplied by price
Future<double> getCartTotal() async {
  final db = await DBHelper.database;
  final rows = await db.rawQuery(
    'SELECT SUM(quantity * price) AS total FROM cart',
  );
  return (rows.first['total'] as num?)?.toDouble() ?? 0.0;
}

// Display cart rows with ListView.builder and total with FutureBuilder<double>.
```

Q17. Student Management – Home, Add, List and Update screens

```
// Home screen navigation examples
ElevatedButton(
  onPressed: () => Navigator.push(context,
    MaterialPageRoute(builder: (_) => const AddStudentScreen())),
  child: const Text('Add Student'),
);

// Insert student from Add Student screen
await DBHelper.insert('students', {
  'name': nameController.text.trim(),
  'email': emailController.text.trim(),
  'course': courseController.text.trim(),
});

// Update student from Update Student screen
await DBHelper.update('students', {
  'name': nameController.text.trim(),
  'email': emailController.text.trim(),
  'course': courseController.text.trim(),
}, student['id'] as int);

// Delete after confirmation
final confirm = await showDialog<bool>(
  context: context,
  builder: (ctx) => AlertDialog(
    title: const Text('Confirm delete'),
    content: const Text('Do you want to delete this student?'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(ctx, false),
        child: const Text('Cancel')),
      TextButton(onPressed: () => Navigator.pop(ctx, true),
        child: const Text('Delete')),
    ],
  ),
);
if (confirm == true) {
  await DBHelper.delete('students', student['id'] as int);
}

// Use DBHelper.getAll('students') to populate the Student List screen.
// Validate required fields with Form and TextFormField before saving.
```

Important implementation notes

• The code blocks are the question-wise core implementations and are designed to be placed inside the corresponding Flutter StatefulWidget methods/build methods. They reuse DBHelper from the beginning of this PDF.

• Create the screens and TextEditingController variables named in each example (for example titleController, nameController, emailController). Dispose controllers in the State class dispose() method.

• After insert, update or delete, reload the data and call setState so the ListView refreshes.

• For a fresh install, the database is created automatically. If you change table definitions during development, uninstall the app or increase the database version and add an onUpgrade migration.

• For real applications, never store passwords as plain text. Use Firebase Authentication or a secure backend with salted password hashing.